

Speaker Diarization Pipeline

A Plain-English Guide to How It All Works

f0.py - speaker_splitter.py - pyannote / HuggingFace

1. The Big Picture

Imagine you record a conversation between two people on a single microphone. You end up with one audio file where both voices are mixed together. The goal of this pipeline is to answer two questions:

- A) Who is speaking at each moment in time?
- B) Can we produce a separate audio file for each speaker?

That is exactly what these two scripts do, in order:

- `f0.py` -- figures out WHO speaks WHEN (this is called diarization).
- `speaker_splitter.py` -- uses that information to carve the audio into one file per speaker, with silence where the other person was talking.

[Tip] Analogy

Think of a court transcript. A stenographer labels every line with the speaker's name. `f0.py` is the stenographer. `speaker_splitter.py` is the clerk who then collects all of one person's lines into a separate document.

2. Step-by-Step Pipeline

1 Load the audio file

load_wave_file() in *f0.py*

The WAV file is read from disk using Python's built-in wave module. The raw bytes are converted to a numpy array of floating-point numbers between -1 and +1, one number per audio sample. If the file is stereo (two channels), only the left channel is kept. This gives us a simple list of numbers that represents the sound wave over time.

2 Run the AI diarization model

Pipeline.from_pretrained() in *f0.py*

This is the core of the whole system. A pre-trained neural network from pyannote (hosted on HuggingFace) analyses the audio and decides, for every fraction of a second, which speaker is

most likely talking. It outputs a list of time segments, each tagged with a speaker label like `SPEAKER_00` or `SPEAKER_01`. The model was trained on thousands of hours of real conversations, so it already understands what makes two voices different without any extra configuration from us.

3 Export a diarization JSON

export_segments_json() in f0.py

The segment list is written to a `.json` file next to the original audio. Each entry records the speaker label, a start time, and an end time in the format `HH:MM:SS,mmm`. This file is the handshake between the two scripts: `f0.py` writes it, `speaker_splitter.py` reads it.

4 Colour the waveform

Visualization block in f0.py

To let you visually verify the result, the audio waveform is drawn in `matplotlib`. The waveform is split into small chunks of 256 samples and each chunk is coloured according to which speaker owns that moment in time. A moving red cursor follows the playback position in real time.

5 Split the audio

create_speaker_audio() in speaker_splitter.py

For each unique speaker found in the JSON, a brand-new silent audio track is created with the same total length as the original. Then, for every segment belonging to that speaker, the corresponding slice of the real audio is copied (overlaid) into the silent track at the correct position. Everything else stays silent. The result is exported as a WAV file named `output-audio-SPEAKER_00.wav`, `output-audio-SPEAKER_01.wav`, and so on.

3. f0.py -- Deep Dive

3.1 Loading audio

Audio files store sound as a sequence of numbers called samples. A sample rate of 16,000 Hz means there are 16,000 numbers per second. The `load_wave_file()` function opens the file, reads all the raw bytes, and converts them from 16-bit integers (the WAV format) to float32 values in the range `[-1, 1]` by dividing by the maximum possible value. Normalisation like this makes downstream processing more stable.

```
audio = np.frombuffer(raw_data, dtype=np.int16).astype(np.float32)
audio /= (np.max(np.abs(audio)) + 1e-8) # normalise to [-1, 1]
```

3.2 Running pyannote

The single most important line in the whole project is:

```
pipeline = Pipeline.from_pretrained("pyannote/speaker-diarization-3.1", token=...)
diarization = pipeline(audio_file, num_speakers=2)
```

`Pipeline.from_pretrained()` downloads (or loads from cache) a complete neural network that was trained by the pyannote team. Internally it runs three stages:

- Voice Activity Detection (VAD) -- find the parts of the audio that contain speech at all, discarding silence and background noise.
- Speaker Embedding -- for each short speech segment, compute a compact numerical fingerprint (a vector of ~256 numbers) that captures the unique characteristics of the voice in that segment.
- Clustering -- group the fingerprints so that segments from the same speaker end up in the same cluster. Each cluster becomes a speaker label.

[Info] What is a speaker embedding?

A speaker embedding is like a voice fingerprint. Two segments from the same person will produce very similar vectors (close together in space). Two segments from different people will produce very different vectors (far apart). The model converts the messy audio signal into this clean mathematical representation so that standard clustering algorithms can separate the speakers.

3.3 Mapping segments to samples

pyannote gives us time segments (e.g. 4.27s - 5.85s = `SPEAKER_00`). But for colouring the waveform,

we need to know the speaker label for every single sample in the audio array. We create a `time_axis` array that maps each sample index to its time in seconds, then use numpy's `searchsorted()` to find where each segment boundary falls in that array, and fill in the speaker label for the whole range.

```
time_axis = np.linspace(0, total_duration, len(audio_array))
for start, end, speaker in segments:
    i0 = np.searchsorted(time_axis, start)
    i1 = np.searchsorted(time_axis, end)
    sample_labels[i0:i1] = speaker_to_idx[speaker]
```

3.4 Timestamp format

The JSON uses the format `HH:MM:SS,mmm` (hours, minutes, seconds, milliseconds). This is the standard subtitle format (SRT) and is also used by `speaker_splitter.py`. The `seconds_to_timestamp()` helper converts a plain float like `74.5` into `00:01:14,500`. The reverse conversion in `speaker_splitter.py` turns that string back into milliseconds (`74500`) so `pydub` can slice the audio precisely.

4. speaker_splitter.py -- Deep Dive

4.1 Reading the JSON

The script takes two command-line arguments: the original WAV file and the diarization JSON. It loads the JSON and collects all unique speaker labels found in the segments list. It then processes each speaker one at a time.

4.2 Building the silent base track

pydub is used to handle audio at a higher level than raw numpy arrays. `AudioSegment.from_wav()` loads the full original audio. `AudioSegment.silent()` creates a new audio object of the same total duration filled with complete silence (all zeros). This silent track is the canvas on which the speaker's audio will be painted.

```
audio = AudioSegment.from_wav(input_audio_path)
silent_audio = AudioSegment.silent(duration=len(audio))
```

4.3 Overlaying the speaker's segments

For each segment in the JSON that belongs to the target speaker, the timestamp is converted to milliseconds, the corresponding slice is cut from the original audio using Python's slice notation (`audio[start_ms:end_ms]`), and then placed onto the silent track at exactly the right position using `overlay()`. Because we started with silence, overlaying a real audio segment simply inserts it without any mixing.

```
for segment in segments_data['segments']:
    if segment['speaker'] == speaker_id:
        start_ms = timestamp_to_milliseconds(segment['start'])
        end_ms = timestamp_to_milliseconds(segment['end'])
        chunk = audio[start_ms:end_ms]
        silent_audio = silent_audio.overlay(chunk, position=start_ms)
```

[!] Important limitation

This approach can only separate speakers in time -- it cannot remove one person's voice from a moment when both people are speaking simultaneously. If Speaker A's voice bleeds into Speaker B's segment (e.g. background talking or overlapping speech), it will still appear in Speaker B's output file. True voice isolation requires a different class of model called a source separator (e.g. SepFormer).

5. The HuggingFace / pyannote Layer

5.1 What is HuggingFace Hub?

HuggingFace Hub is a platform where researchers publish pre-trained AI models. Think of it like GitHub, but specifically for neural networks. A model is a file (or set of files) containing millions of numbers that were learned during a long training process on large datasets. By downloading a model from the Hub, you skip the training step entirely and use the results of someone else's months of compute.

- `Pipeline.from_pretrained(model_id)` -- downloads the model files to your local cache (`~/.cache/huggingface/`) on the first run, then reuses them.
- The token (`hf_...`) -- proves to HuggingFace that you agreed to the model's terms of use. Some models (like pyannote's) are gated: you must request access before the Hub will serve the files to you.
- After the first download everything runs fully offline -- no internet required.

5.2 What pyannote actually runs

The pyannote speaker-diarization-3.1 pipeline is itself a composition of three smaller neural networks, each a separate HuggingFace model:

- `pyannote/segmentation-3.0` -- a transformer that runs on short windows of audio and outputs a probability for each window: is there speech here, and if so, how many speakers?
- `pyannote/embedding (wespeaker-voxceleb-resnet34-LM)` -- a ResNet-based network that converts a speech segment into a 256-dimensional embedding vector.
- `pyannote/speaker-diarization-community-1` -- contains a PLDA model, a classical (non-neural) scoring method that refines how embeddings are compared to improve accuracy on real-world audio.

5.3 What a ResNet is (in plain English)

A ResNet (Residual Network) is a type of deep neural network originally designed for image recognition. For audio, the raw waveform is first converted to a spectrogram (a 2-D image where one axis is time and the other is frequency). The ResNet then processes this image, layer by layer, learning to detect patterns -- pitch, timbre, speaking rhythm -- that are unique to a particular voice. The final layer outputs the 256-number embedding that summarises everything the network learned about that voice.

[Note] Why 256 numbers?

256 is the dimensionality of the embedding space. It is a design choice: large enough to capture the richness of a human voice, small enough to cluster efficiently. Two embeddings from the same speaker will have a cosine similarity close to 1.0; two from different speakers will be much lower.

5.4 PLDA scoring

PLDA (Probabilistic Linear Discriminant Analysis) is a statistical model that learns, from labelled training

data, how much natural variation exists within a single speaker versus across different speakers. It adjusts the raw embedding comparisons to be more robust to things like microphone differences, background noise, and the speaker's emotional state. It is stored as a matrix file (`xvec_transform.npz`) and applied after the neural embedding step.

6. How the Algorithm Separates Speakers -- Step by Step

This chapter goes inside the black box. When you call `pipeline(audio_file)`, here is exactly what happens internally, in order.

Step 1 -- From waveform to spectrogram

The raw audio is a one-dimensional list of numbers -- one pressure value per sample (16,000 per second). Neural networks cannot do much with this directly. The first transformation converts it into a mel spectrogram: a 2-D grid where the horizontal axis is time and the vertical axis is frequency (pitch). The value at each cell represents how much energy exists at that frequency at that moment in time.

The 'mel' scale compresses high frequencies and expands low ones, mimicking how the human ear perceives pitch. This makes the spectrogram much more useful for voice analysis than a raw frequency grid would be.

[Tip] Think of it like sheet music

In sheet music, time flows left to right and pitch flows bottom (low notes) to top (high notes). A mel spectrogram is the same idea, but instead of note symbols it shows the energy (volume) at every pitch at every moment. Every neural network in this pipeline reads this 'image' rather than the raw waveform.

Step 2 -- Voice Activity Detection (VAD)

Before doing any speaker identification, the pipeline needs to know which parts of the audio actually contain speech. This is done by the `pyannote/segmentation-3.0` model, a transformer-based neural network.

The model slides a window (~10 seconds wide) across the spectrogram and for each window predicts, frame by frame:

- Is there any speech here at all, or just silence / background noise?
- If yes, how many speakers appear to be active simultaneously (1 or 2+)?
- Where does the current speaker seem to change?

The output is a timeline of speech segments with silence removed. This step is critical: it means the expensive embedding network in Step 3 only runs on frames that actually contain someone speaking, not on silence or music or background noise.

Step 3 -- Speaker Embedding: building a voice fingerprint

This is the heart of the algorithm. For each speech segment identified in Step 2, the `wespeaker` ResNet-34 model computes a speaker embedding -- a vector of 256 numbers that acts as a compact fingerprint of the voice in that segment.

Internally, the ResNet works like this:

- The segment's mel spectrogram is treated exactly like an image and fed into a 34-layer deep convolutional neural network.
- Early layers detect simple patterns: edges in the frequency domain, basic pitch contours, short bursts of energy.
- Middle layers combine these into more complex patterns: formant shapes (the resonance frequencies of the vocal tract), speaking rhythm, breath patterns.
- Deep layers capture the most abstract characteristics: the overall acoustic 'texture' that is unique to a given person's voice.
- A pooling layer collapses the time dimension -- it computes the mean and standard deviation of the activations across all time steps -- giving a single fixed-size vector regardless of how long the segment is.
- A final linear projection reduces this to exactly 256 numbers.
- The vector is L2-normalized: scaled so its total length equals 1.0. This allows similarity to be measured purely by direction, not magnitude.

[Info] Why does L2-normalization matter?

After normalization, all embedding vectors sit on the surface of a 256-dimensional sphere. The angle between two vectors on this sphere is all that matters for comparison. Two segments from the same speaker will produce vectors pointing in nearly the same direction (cosine similarity close to 1.0). Two segments from different speakers will point in very different directions (cosine similarity much lower). This geometry is what makes clustering work reliably.

Step 4 -- PLDA: smarter similarity scoring

Raw cosine similarity between embeddings works, but it is noisy. The same person sounds different when excited versus calm, or when speaking close to the microphone versus from across the room. PLDA (Probabilistic Linear Discriminant Analysis) fixes this.

PLDA is a statistical model trained on a large labelled dataset of recordings where the speaker identity of every segment is known. From this data it learns two things:

- Within-speaker variability: how much do two embeddings from the SAME person naturally differ from each other due to recording conditions, emotion, or speaking style?
- Between-speaker variability: how much do embeddings from DIFFERENT people differ on average?

It uses this knowledge to transform the embedding space so that within-speaker differences shrink and between-speaker differences grow. The result is a PLDA score between any two segments -- a number that is much more reliable than raw cosine similarity for deciding whether two segments come from the same person.

The transformation is stored as a matrix (xvec_transform.npz) and applied to every embedding before any comparisons are made. It is a classical linear algebra operation, not a neural network, so it is very fast.

Step 5 -- Agglomerative Hierarchical Clustering

Now we have N embeddings (one per speech segment) and PLDA-adjusted similarity scores between every pair. The clustering algorithm groups them into speaker identities:

- Start: treat every segment as its own cluster (N clusters total).
- Compute the similarity between all pairs of clusters.
- Find the two most similar clusters and merge them into one.
- Recompute similarities involving the new merged cluster.
- Repeat until the number of clusters equals num_speakers (2 in our case).

This is called 'agglomerative' (bottom-up) clustering because it starts with many small clusters and merges upward, as opposed to 'divisive' (top-down) approaches that start with one cluster and split. The merging criterion -- how to compute the similarity of a merged cluster to others -- uses average linkage: the similarity between two clusters is the average of all pairwise similarities between their members.

[Tip] Analogy: sorting strangers into groups

Imagine you have 50 short audio clips and you want to sort them into two piles by speaker. You start by listening to the two most similar clips and put them in the same pile. Then you keep finding the most similar remaining pair and grouping them. Eventually you end up with exactly two piles -- that is agglomerative clustering.

Step 6 -- Resolving overlaps

After clustering, the raw output (speaker_diarization) may contain overlapping segments -- moments where two speakers are both active simultaneously. This is realistic (people do talk over each other), but speaker_splitter.py needs clean non-overlapping segments.

The exclusive_speaker_diarization pass resolves this: for every overlapping moment it picks the speaker with the higher confidence score and assigns the entire frame to them. The other speaker's contribution at that moment is discarded. The result is a clean timeline where every moment in time belongs to exactly one speaker or to silence.

Why it sometimes fails

Understanding the algorithm makes it easy to predict when it will struggle:

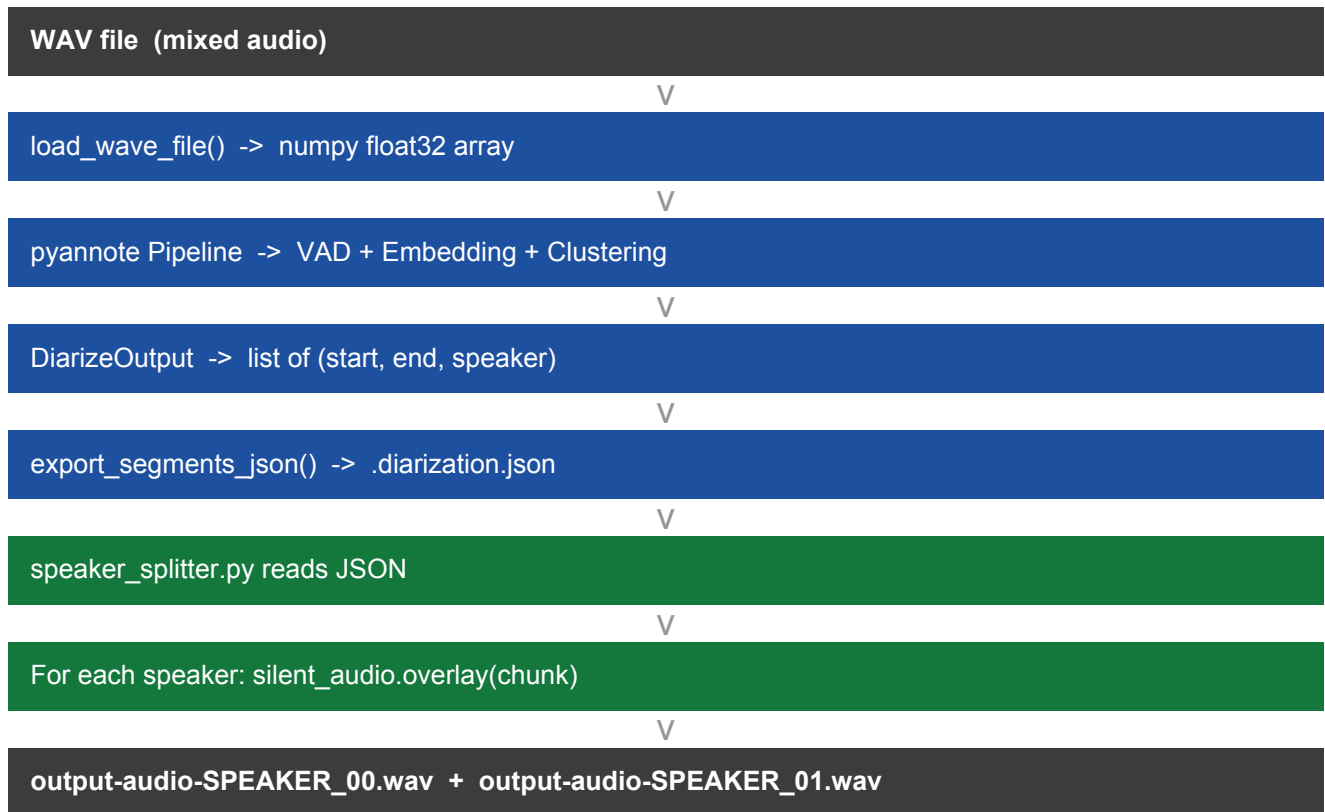
- Similar voices: if two speakers have similar pitch, timbre, and rhythm, their embeddings will be close together in the 256-dimensional space and clustering may merge them into one cluster.
- Short segments: the ResNet needs at least a few seconds of speech to produce a reliable

embedding. Very short utterances (under ~1 second) produce noisy fingerprints.

- Poor audio quality: microphone bleed, heavy reverb, or background music alter the spectrogram in ways that shift embeddings away from their true position, confusing the PLDA scoring.
- One speaker dominates: if one person speaks 90% of the time, the clustering may not gather enough evidence for the second speaker's cluster and may split the dominant speaker into two clusters instead.
- More speakers than expected: if `num_speakers=2` is passed but three people speak, the algorithm is forced to squeeze three voices into two clusters, guaranteeing at least some mislabeling.

7. Data Flow at a Glance

Here is how data moves through the system from start to finish:



8. Glossary

Diarization

The process of partitioning an audio stream into segments according to who is speaking. The name comes from 'diary' -- a record of who said what when.

Voice Activity Detection (VAD)

A pre-processing step that identifies which parts of an audio file contain speech and which are silence or noise. This avoids wasting computation on non-speech frames.

Speaker Embedding

A fixed-size vector of numbers that encodes the acoustic characteristics of a speaker's voice. Similar voices produce similar vectors.

Cosine Similarity

A measure of how similar two vectors are, regardless of their length. A value of 1.0 means they point in the same direction (identical voices); 0.0 means they are completely unrelated.

Spectrogram

A 2-D visual representation of audio where the x-axis is time, the y-axis is frequency, and the brightness/colour shows the energy at each frequency at each moment. Neural networks process spectrograms like images.

ResNet (Residual Network)

A deep neural network architecture that uses skip connections to allow gradients to flow easily during training. Originally designed for image classification, widely adapted for audio feature extraction.

PLDA (Probabilistic Linear Discriminant Analysis)

A classical statistical model that refines similarity scores between speaker embeddings by accounting for within-speaker variability.

pydub

A Python library that provides a high-level interface for audio editing: slicing, concatenating, overlaying, and exporting audio segments.

Sample Rate

The number of audio samples recorded per second. 16,000 Hz (16 kHz) is standard for speech processing. Higher rates capture more detail but require more storage and computation.

HuggingFace Hub

An online platform for sharing and downloading pre-trained machine learning models. Acts as a repository (like PyPI or npm) but for neural networks.

Gated Model

A HuggingFace model that requires you to log in and accept terms of use before you can download it. Access is granted per-user.

Source Separation

A different (harder) problem: given a mixed audio signal, recover the individual source signals (one per speaker). Unlike diarization, it produces audio where the other speaker is truly removed, not just silenced.